Algorithm for Release 3

Course: ICS0017 Fundamentals of C++ Programming

Release 3 is designed to be focused and achievable. It does not add new modules or layers. It makes your existing design safer and clearer. The goal is quality: validation, preconditions and postconditions, behavioral modeling, and clear exception handling.

Scope and Success Criteria

You continue with the same architecture and class names from Release 2: UI → Logic → Repository. No new layers or modules. The Repository represents the Data layer from Release 2. The structure remains the same, only the focus of Release 3 changes from building to behavior.

Success in Release 3 means:

• You identified and wrote validation rules for key methods using preconditions and postconditions.

• You created two behavioral UML diagrams (Activity and Sequence) for the same operation.

• You defined a clear exception policy using UI, Logic, and Repository as locations of throw/catch.

Main Rules

1) Reuse the same class and method names from Release 2.

2) Do not introduce new layers or extra services.

3) All exceptions are caught at the UI layer, which communicates the message to the user.

5) Keep messages clear and non-technical for the user interface.

Step-by-Step Plan

Step 1

Review your Release 2 structure.

Confirm that your three-layer architecture is in place and class names are consistent.

Step 2

Select one core operation for Release 3.

Choose a single scenario in your domain (for example, confirmReservation, processOrder, registerUser, processPayment). This is the operation you will model in both diagrams and reference in validation and exception tables.

Note. One Logic Explained

In this release, each team must focus on **one logic**, one complete operation that goes through all three layers: **UI → Logic → Repository (Data Layer)**.

The Repository here represents the **Data layer** — the place where information actually lives (a file, a database, or an in-memory structure).
Its job is to **provide or deny** data and, if necessary, signal an error — it never decides what to do with it.
The Logic layer interprets the rules, and the UI reports the result to the user.

One logic means one continuous story:
it starts with a user action in the UI, continues through the Logic layer where validations and calculations happen, and ends in the Repository layer, where data is confirmed or an exception is raised.

It's not about one class or one file, it's about **one flow of responsibility** that can succeed or fail.
All validations, computations, and exceptions related to this single operation belong to the same flow and must appear in your table and later in both UML diagrams.

In this document, we look at the same logic from three perspectives:

- **Section 13** Validation and Exception Rules

- **Section 14** Behavioral UML Diagrams (Activity and Sequence)
- **Section 15** Error and Exception Policy

This way, your Release 3 work shows a clear, living process where each layer plays its role: **UI handles and reports**, **Logic validates and throws**, and **Repository confirms or signals problems**.

Example Logic Used in This Document

Operation: convert(amount, from, to)
(All validations, diagrams, and exception policies below refer to this operation.)

The operation converts a user-entered amount from one currency into another using an exchange rate from the Repository.
It demonstrates the complete behavioral chain - validation, computation, and controlled error handling.

Aspect	**Description**
Inputs (UI)	amount, from, to
Preconditions (Logic)	amount > 0; currency codes valid
Data check (Repository)	rate exists for (from, to)
Postconditions (Logic)	result = amount × rate
Exceptions	InvalidInput (UI) – empty or invalid fields InvalidData (Logic) – amount ≤ 0 or unknown codes MissingRate (Repository) – rate not found
Normal flow	UI → Logic.validate → Repository.getRate → Logic.compute → UI displays result
Error flow	Exception thrown in Logic or Repository → propagated to UI → user-friendly message (“This currency pair is not supported.”)

How it connects to your Release 3 deliverables

Section	Focus	What to include
13 — Validation & Exception Table	Preconditions, postconditions, exception list	Mark validation level (UI / Logic / Repository) and where each exception is thrown and caught (UI).
14 — Behavioral Models	Activity and Sequence diagrams	Show the same operation with green (normal) and red (error) paths.
15 — Exception Policy	Exception messages and actions	Define user messages and default actions (message / stop / retry).

This ensures that every rule from your table appears in your diagrams,
and every exception from your diagrams is documented in your policy.
Your Release 3 documentation tells **one coherent story**, the same logic, shown in three complementary ways.

Step 3

Write Validation Rules (DLD Section 13).

For each key method involved in the operation, write preconditions and postconditions and indicate the validation level (UI, Logic, Repository). Use a single sentence explanation for why validation happens at that layer.

Step 4

Build Behavioral UML (DLD Section 14).

Create an Activity Diagram and a Sequence Diagram for the same operation. Mark decisions as preconditions and end nodes as postconditions. In Sequence, show UI → Logic → Repository and exception propagation back to UI.

Step 5

Define Error and Exception Policy (DLD Section 15).

List exceptions that can happen in this operation. For each, specify where it is thrown, where it is caught, the user message, and the default action (message, stop, retry).

Step 6

Update Revision History (DLD Section 16).

Record the changes you have made in Release 3.

Section 13 — Validation Rules (Examples)

Use these examples as a pattern. Replace with your domain terms and your actual classes and methods from Release 2.

The following table lists the validation and exception rules for the selected operation convert(amount, from, to).

Each rule represents a design constraint that must be checked before or during execution.

Every validation later appears as a decision node in the Activity Diagram and as a message or condition in the Sequence Diagram.

Class / Method	**Preconditions**	**Postconditions**	**Validation Level**	**Explanation**
ConsoleUI::readInput()	Fields “amount”, “from”, “to” are entered	Input values captured	**UI**	Checks that all fields are provided and syntactically correct
ExchangeManager::convert(amount, from, to)	amount > 0, currency codes valid	result = amount × rate	**Logic**	Validations executed before calling the Repository
CurrencyRepository::getRate(from, to)	Rate pair exists in data source	Returns exchange rate or throws MissingRate	**Repository (Data)**	Confirms data availability; signals if rate is missing
ExchangeManager::convert()	Rate retrieved successfully	Returns numeric result	**Logic**	Confirms successful flow and prepares result for UI

Each precondition from this table appears as a decision in the Activity Diagram;
each exception here is represented as a red branch in the diagrams below.

Patterns for writing preconditions and postconditions:
• Precondition pattern: “parameterName satisfies constraint” (for example, amount > 0; date is in the future; id format is valid).
• Postcondition pattern: “state or result satisfies constraint” (for example, result > 0; record created; status updated).

Section 14 — Behavioral Models (Activity and Sequence)

Create two diagrams for the same operation.
This section visualizes the same operation, convert(amount, from, to),
from two behavioral perspectives.

- **Activity Diagram** shows the logical flow of validation, data retrieval, and computation,
  including red branches for exception handling.
- **Sequence Diagram** shows the interaction between layers **UI → Logic → Repository**, including normal and exceptional paths.

Build two diagrams for the same operation:

1. Normal flow - validation succeeds, rate found, result returned.
2. Error flow (red path) - validation fails or rate missing; exception propagated to UI.

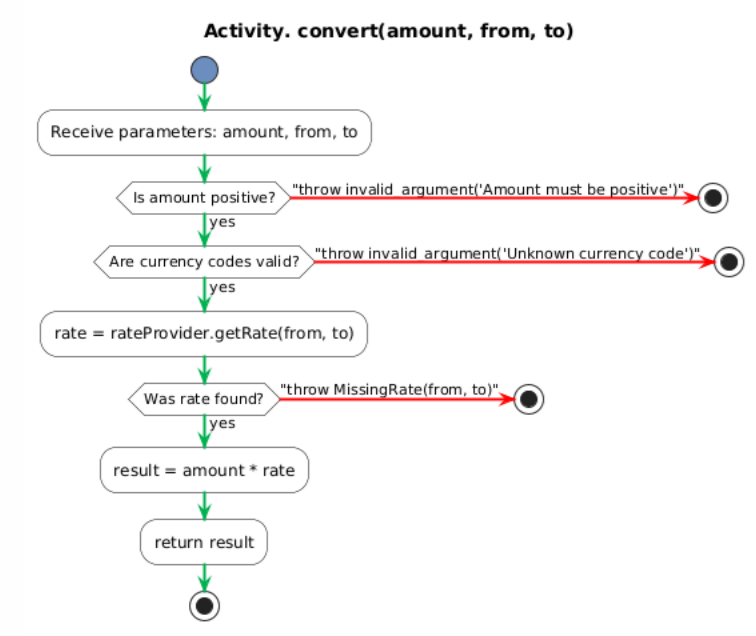
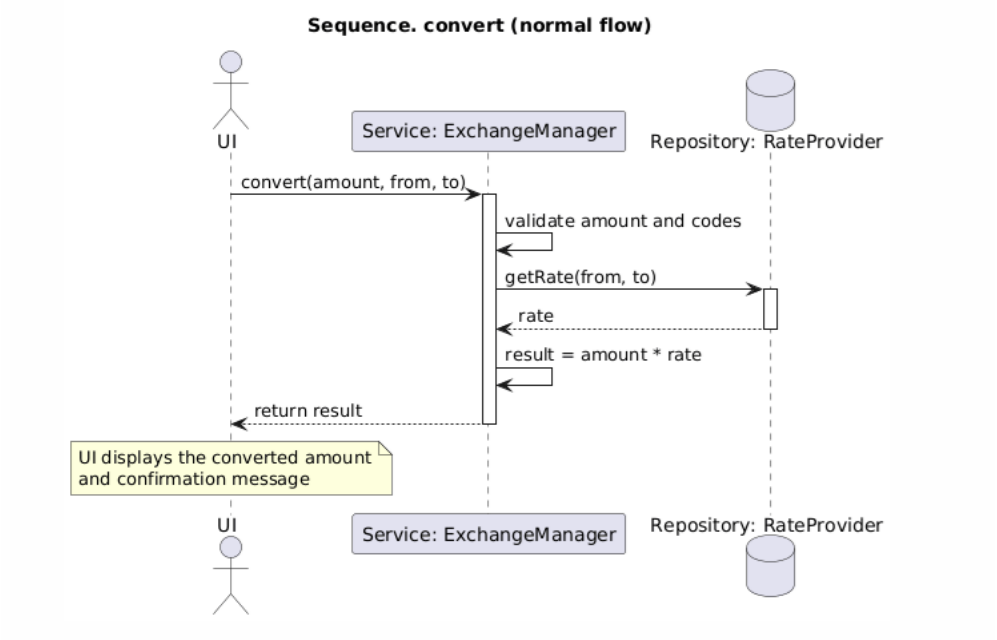


Figure 14.1 *Activity Diagram. convert(amount, from, to)*

-



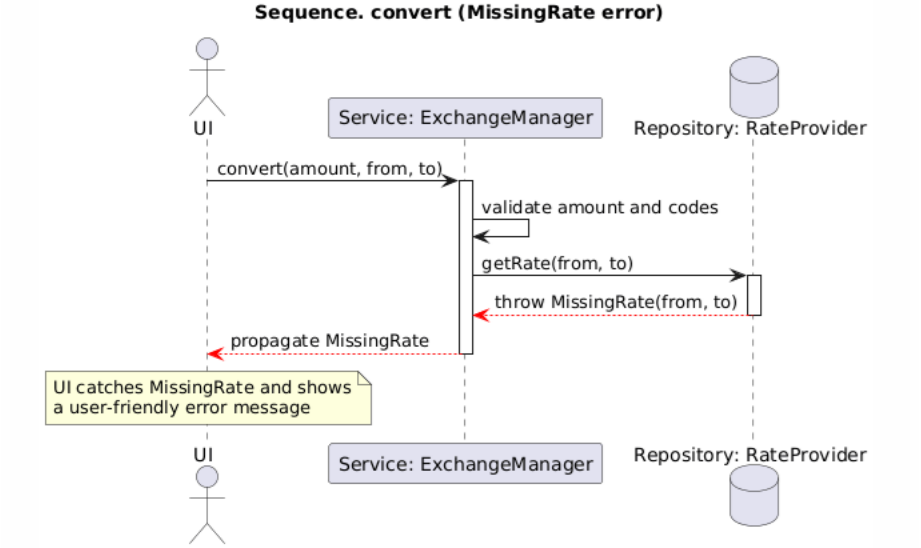


Figure 14.2 *Sequence Diagram UI → Logic → Repository (convert flow)*

Section 15 Error and Exception Policy (Examples)

The table below defines how each exception type is thrown, caught, and reported to the user. All exceptions listed here appear as red branches in the Activity and Sequence diagrams. No logging system is required (this topic will be covered in ICS0025 Advanced C++).

Exception Type	**Thrown By (Layer / Class)**	**Caught At (Layer)**	**User Message**	**Default Action**
InvalidInput	UIConsole input validation	**UI**	"Please enter all required fields."	Show message → ask user to re-enter
InvalidData	Logic — ExchangeManager guard clause	**UI**	"Amount must be positive or currency not recognized."	Show message → stop operation
MissingRate	Repository — CurrencyRepository	**UI**	"Exchange rate not found for selected pair."	Show message → stop operation

Exception Type	Thrown By (Layer / Class)	Caught At (Layer)	User Message	Default Action
DataSourceError	Repository — FileService / Data Provider	**UI**	“Data source unavailable or corrupted.”	Show message → stop operation

All exceptions in this table correspond to red branches in Figure 14.1 and 14.2.

Release 3 is not about more code, it’s about more clarity.

You are proving that your design can handle both success and failure predictably.